Two Error Models

Document Number:	D4054R0
Date:	2026-03-12
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. The Partition

- 2.1 Infrastructure Operations
- 2.2 I/O Operations
- 2.3 The OS Delivers Pairs

3. The Infrastructure Error Model

4. The I/O Error Model

5. The Channel Split

- 5.1 Known Mappings

6. Three Defenses, Three Concessions

- 6.1 Accept the Information Loss
- 6.2 Route Everything Through `set_value`
- 6.3 Classify Errors Into Channels
- 6.4 The Fork

7. Domain Boundary

8. Conclusion

References

Abstract

Asynchronous operations partition naturally into two classes based on their error semantics. Infrastructure operations have binary outcomes: the operation executed or it did not. I/O operations produce compound results: a status classification and associated data, always paired. This paper identifies the partition and examines the sender three-channel model against it. This paper is informational and proposes no action.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

1. Disclosure

The author developed P4007R0[6] (“Senders and Coroutines”) and publishes coroutine-native I/O libraries. The analysis below holds regardless of whether any alternative design exists. Every claim is sourced to published committee papers, OS specifications, or cited implementations.

2. The Partition

Asynchronous operations fall into two classes based on what their errors mean.

2.1 Infrastructure Operations

Launch, schedule, dispatch, allocate, enqueue. The operation either executes or it does not. An error means the postcondition was violated.

Operation	Success	Failure
GPU kernel launch	Kernel running	Launch failed
Thread pool submit	Task queued	Pool exhausted

Operation	Success	Failure
Timer arm	Timer armed	Resource limit
Memory allocate	Block returned	Allocation failed
Mutex acquire	Lock held	Deadlock / timeout

Every row is binary. Success is one thing. Failure is another. There is no middle ground, no partial outcome, no compound result. The operation either satisfied its postcondition or it did not.

2.2 I/O Operations

Read, write, connect, accept, send, receive. The operation always completes. The result is a classification of what happened, paired with associated data.

Operation	Result
Read	`(status, bytes_transferred)`
Write	`(status, bytes_written)`
Connect	`(status)`
Accept	`(status, peer_socket)`

The status code is not evidence of a postcondition violation. It is the postcondition. `read(fd, buf, n)` promises to report what happened on the descriptor. Every status code fulfills that promise:

error_code	bytes	What happened	Postcondition violated?
success	N	Full read	No
success	M < N	Partial read	No
`EOF`	0	Peer closed	No
`ECONNRESET`	0	Peer reset	No
`EWOULDBLOCK`	0	Try again	No
`EBADF`	0	Invalid fd	Yes

Only `EBADF`-class errors - invalid file descriptor, bad address, not a socket - are postcondition violations. Every other outcome is the operation correctly reporting the state of the world.

2.3 The OS Delivers Pairs

This is not a design preference. It is how operating systems report I/O completions:

- **io_uring.** Each completion queue entry (CQE) carries `res` (byte count or negative errno) and `flags`. One structure. One dequeue. The kernel does not route success and failure to different queues.
- **IOCP.** `GetQueuedCompletionStatus` returns a `BOOL`, an `lpNumberOfBytesTransferred`, and an `lpOverlapped`. The error and the byte count arrive together in one call.
- **POSIX.** `read()` returns `ssize_t`. On success, the byte count. On failure, `-1` with `errno` set. Both values are available at the same call site.

Three OS families, three different APIs, the same shape: status and data arrive as a pair because they are a single result.

---

3. The Infrastructure Error Model

The sender three-channel model assigns semantic meaning to each channel:

- `set_value(args...)` - the postcondition was satisfied; here are the results
- `set_error(err)` - the postcondition was violated; here is why
- `set_stopped()` - the operation was cancelled before attempting the postcondition

For infrastructure operations, this mapping is unambiguous:

```
// Timer sender: binary outcome
auto timer_sender = schedule_at(scheduler, deadline);
// Success: set_value()       - timer expired
// Failure: set_error(ec)     - resource exhaustion
// Cancel:  set_stopped()     - cancelled before expiry
```

```
// Thread pool submit: binary outcome
auto submit_sender = on(pool.get_scheduler(), work);
// Success: set_value(result) - work executed
// Failure: set_error(ec)     - pool rejected
// Cancel:  set_stopped()     - cancelled before dispatch
```

Channel assignment is deterministic. No information is lost. No adaptation is required. `then` sees the result. `upon_error` sees the error. `upon_stopped` sees the cancellation. The channels compose with generic algorithms: `retry` retries on `set_error`, `when_all` cancels siblings on failure, `upon_error` routes errors to handlers.

The three-channel model is correct for this class. `std::execution` serves it well.

---

4. The I/O Error Model

For I/O operations, the postcondition is: "return the outcome classification and associated data." This postcondition is always satisfied. The operation always reports what happened.

The status code is not a failure signal. It is a vocabulary:

- `ECONNRESET` - the peer reset the connection
- `EPIPE` - the write end is closed
- `ETIMEDOUT` - the peer did not respond within the timeout
- `EWOULDBLOCK` - nothing available now; try again
- `EOF` - the peer closed the connection gracefully

Each requires different application handling. None indicates that the operation failed to operate. `ECONNRESET` means "fatal, abort the transaction" in an HTTP request handler, but "done, expected closure" in a long-polling connection that the server intentionally drops. The same error code, classified differently depending on the protocol state the application holds.

Partial results are normal. A `write` that sends 500 of 1,000 bytes before `ECONNRESET` produces `(ECONNRESET, 500)`. Both values are needed. The 500 tells the application how much of the message was acknowledged. The status tells it why the rest was not.

Chris Kohlhoff identified this in P2430R0[2] ("Partial success scenarios with P2300," 2021):

> *"Due to the limitations of the `set_error` channel (which has a single 'error' argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel."*

POSIX has modeled I/O this way since 1988[9]. Asio has modeled it this way since 2003. Every language runtime that provides async I/O - Go, Rust, Python, C#, Java - delivers compound results. This is not a novel observation. It is forty years of convergent practice.

5. The Channel Split

When a sender-based I/O operation completes, it must choose a channel. The byte count and the status code - which the OS delivered as a pair - are now split:

```
async_read(socket, buffer)
    | then([](size_t n) {
          // byte count visible
          // error_code invisible
      })
    | upon_error([](std::error_code ec) {
          // error_code visible
          // byte count invisible
      });
```

`then` and `upon_error` are mutually exclusive execution paths. The programmer cannot inspect both values at the same call site. The I/O result was a pair. The model split it.

To recover the pair, the programmer must defeat the channels:

```
async_read(socket, buffer)
    | into_expected
    | then([](std::expected<size_t, std::error_code> result) {
          // both visible again
      });
```

This works. But the programmer paid the cost of the channel model - the split, the adaptor, the wrapping - to undo the channel model. The channels are not helping. They are an obstacle being worked around. `let_error` is the same pattern: catch the error, re-emit through `set_value`. Channel laundering.

Consider a partial write. The application asks to write 1,000 bytes. The kernel accepts 500 before the connection dies. `set_error(ECONNRESET)` discards the 500. The application needs that number to know what was acknowledged by the peer - whether to retry, what offset to resume from, whether the transaction can be salvaged. The channel model destroyed it.

5.1 Known Mappings

Two known attempts to solve the channel assignment problem for I/O illustrate the structural difficulty. Neither involves coroutines. Both operate at the receiver level: which of `set_value`, `set_error`, `set_stopped` does the I/O sender call when the kernel completion arrives?

The default mapping. beman.net[7], the reference sender-based networking implementation:

```
if (!ec)
    set_value(std::move(rcvr), n);
else
    set_error(std::move(rcvr), ec);
```

All non-zero error codes go to `set_error`. The byte count is discarded unconditionally. Total information loss on any error.

The refined mapping. Peter Dimov proposed a convention in P4007R0[6] (Section 3.6) that preserves partial results by discriminating the channel based on the byte count and error code category:

Completion	Channel
`(eof, n)` for any n	`set_value(eof, n)`
`(canceled, n)` for any n	`set_stopped()`
`(ec, 0)` where ec is not eof	`set_error(ec)`
`(ec, n)` where n > 0	`set_value(ec, n)`

This is the only known mapping that preserves partial results on the error path. Dimov characterized it as "ad hoc."[6] The mapping requires semantic knowledge of specific error conditions: EOF is distinguished from cancellation, which is distinguished from all other errors. It is domain-specific I/O logic embedded in the channel routing.

Even this mapping has costs. Zero-byte errors lose data - they go to `set_error` with no byte count. The classification is context-free: the I/O layer decides which channel before the application can inspect the result. `ECONNRESET` is classified irrevocably at the I/O layer, before the protocol handler - the only code that knows whether `ECONNRESET` is fatal or expected - has a chance to see it.

Both mappings demonstrate the same structural problem: any function from `(error_code, size_t)` to `{set_value, set_error, set_stopped}` must either lose data, embed domain-specific classification at the wrong layer, or bypass the channels entirely. The problem is not the mapping. The problem is that the mapping

is required at all.

---

6. Three Defenses, Three Concessions

Three positions are available to defend the three-channel model for I/O. Each concedes the thesis.

6.1 Accept the Information Loss

Argue that the byte count does not matter when an error occurs. `set_error(ECONNRESET)` is sufficient; the partial transfer count is unneeded.

This contradicts POSIX semantics, where `write()` returns the number of bytes written even when a subsequent call would fail. It contradicts Asio's `(error_code, size_t)` convention, which has delivered both values for twenty years. It contradicts network programming practice across every language and framework. Partial writes are normal. The byte count is how the application knows what was acknowledged.

6.2 Route Everything Through `set_value`

Call `set_value(error_code, bytes)` for all outcomes. The pair stays intact. No information loss.

But `set_error` and `set_stopped` now serve no purpose for I/O senders. Generic algorithms that dispatch on channels - `retry`, `when_all`, `upon_error` - cannot distinguish I/O success from I/O failure because both arrive through `set_value`. The three-channel model reduces to one channel with a structured result. The channels are not wrong. They are irrelevant.

6.3 Classify Errors Into Channels

Route "routine" errors like `ECONNRESET` through `set_value` and "exceptional" errors like `ENOMEM` through `set_error`. This is the refined mapping from Section 5.1 taken further.

This requires a taxonomy of error codes that does not exist in POSIX, Windows, or any OS API. It redefines `set_value` from "the operation succeeded" to "here is a result that might contain an error." And the classification must happen at the I/O layer, before the application - the only code with enough context to classify correctly - has seen the result. `ECONNRESET` is fatal in one protocol and expected in another. No context-free classification can get this right.

6.4 The Fork

Each position concedes the thesis:

- (a) loses I/O data that applications need
- (b) makes the channels irrelevant for I/O
- (c) redefines what the channels mean and classifies at the wrong layer

There is no fourth position. The three-channel model either loses data, becomes unused, or changes its semantics when applied to I/O.

---

7. Domain Boundary

The finding is not that the three-channel model is wrong. It is that the model has a domain.

Domain	Error model	Three-channel model fits?
GPU dispatch	Infrastructure	Yes
Thread pool	Infrastructure	Yes
Timer	Infrastructure	Yes
Allocator	Infrastructure	Yes
Read / Write	I/O	No
Connect/Accept	I/O	No
DNS resolve	I/O	No

`std::execution` was built at NVIDIA for compile-time work graphs and GPU dispatch[8]. Those are infrastructure operations with binary outcomes. The model is correct for its design domain.

Dietmar Kühl - the author of P3552R3[5] (`std::execution::task`) and P2762R2[3] (sender/receiver networking) - described his own I/O error dispatch mechanism on the LEWG reflector (March 12, 2026): "My answer does somewhat appall me, especially having created this - er - solution!" He was demonstrating the four different syntactic mechanisms required to dispatch to the three channels from a coroutine. The task author's own assessment of what happens when the infrastructure error model meets I/O.

---

8. Conclusion

Two error models exist. They are not a matter of taste. They follow from the structure of the operations.

Infrastructure operations have binary postconditions. The three-channel model maps them cleanly. I/O operations produce compound results where the status code and the associated data are inseparable. The three-channel model cannot represent them without losing data, bypassing the channels, or redefining what the channels mean.

The partition is the same one Chris Kohlhoff identified in 2021[2]. It is the same one every OS kernel enforces by delivering status and byte count as a single completion. It is the reason Asio delivers `(error_code, size_t)` as a pair. It is not new. It is forty years old.

References

1. P2300R10 - ''std::execution'' (Michał Dominiak et al., 2024). https://wg21.link/p2300r10
2. P2430R0 - ''Partial success scenarios with P2300'' (Chris Kohlhoff, 2021). https://wg21.link/p2430r0
3. P2762R2 - ''Sender/Receiver Interface For Networking'' (Dietmar Kühl, 2023). https://wg21.link/p2762r2
4. P3570R2 - ''Optional variants in sender/receiver'' (Fabio Fracassi, 2025). https://wg21.link/p3570r2
5. P3552R3 - ''Add a Coroutine Task Type'' (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
6. P4007R0 - ''Senders and Coroutines'' (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4007r0
7. bemanproject/net - Sender/receiver networking library. https://github.com/bemanproject/net
8. stdexec - NVIDIA reference implementation of std::execution. https://github.com/NVIDIA/stdexec
9. IEEE Std 1003.1 - POSIX `read()` / `write()` specification. https://pubs.opengroup.org/onlinepubs/9799919799/